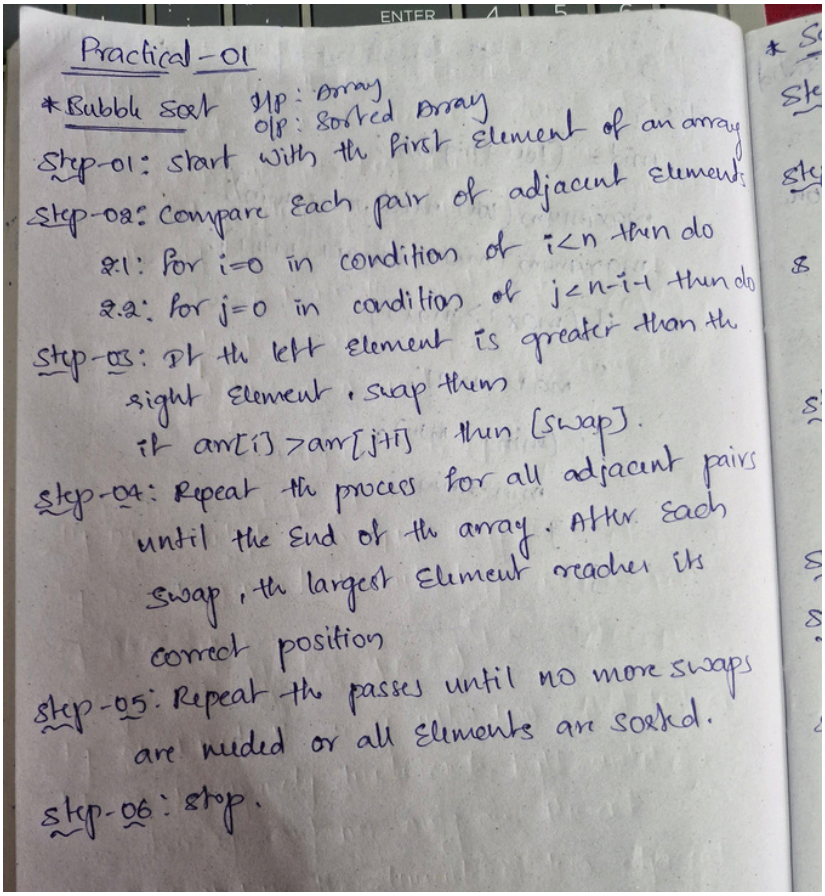
Practical: 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
29/07/2026	

Aim: ToImplement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort,Merge sort and Quicksort.

1.Bubblesort:

Algorithm:



Program:

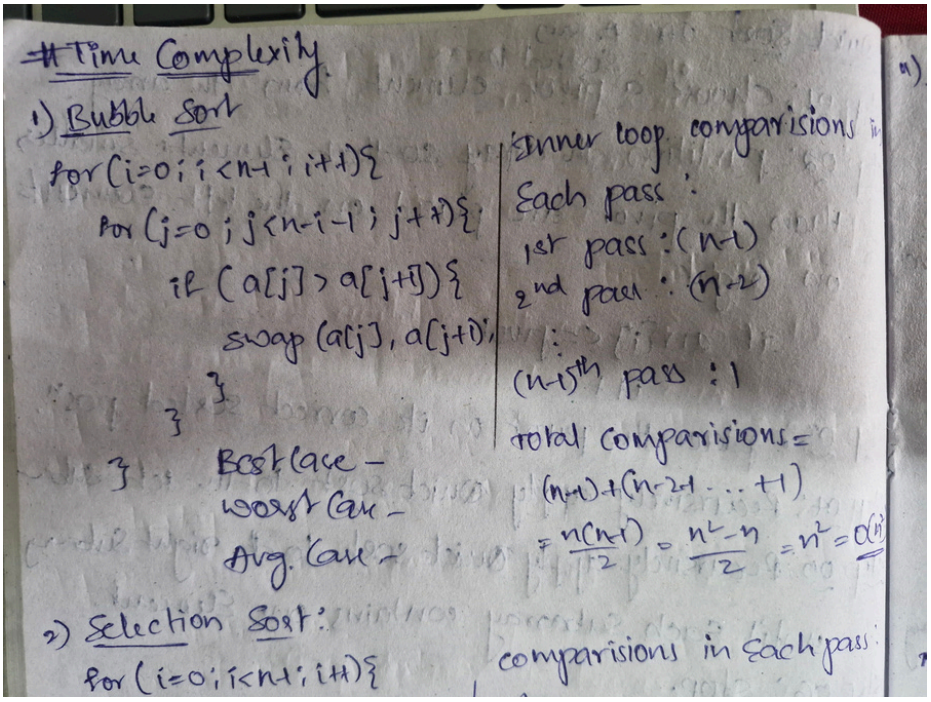
```
#include <iostream>
#include <vector>

using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
```

```
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;

    vector<int> arr(size);
    cout << "Enter " << size << " integers: ";
    for (int i = 0; i < size; ++i) {
        cin >> arr[i];
    }

    bubbleSort(arr);

    cout << "Sorted array: ";
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

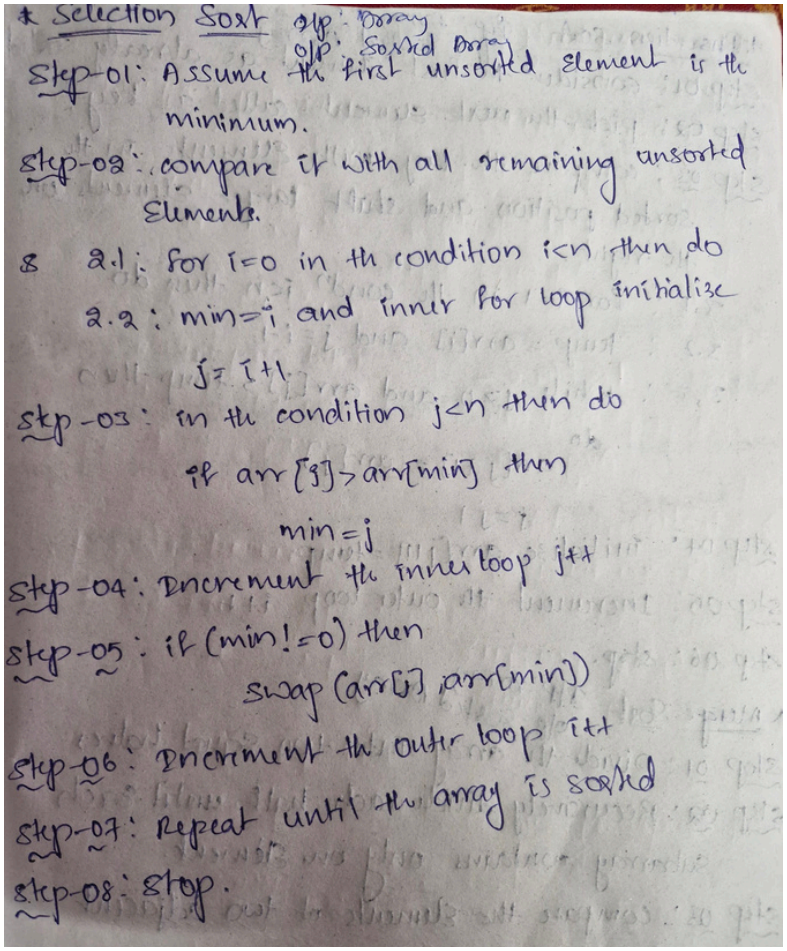
```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 5
Enter 5 integers: 5 1 10 3 2
Sorted array using bubblesort : 1 2 3 5 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time Complexity:



2.Selection sort:

Algorithm:



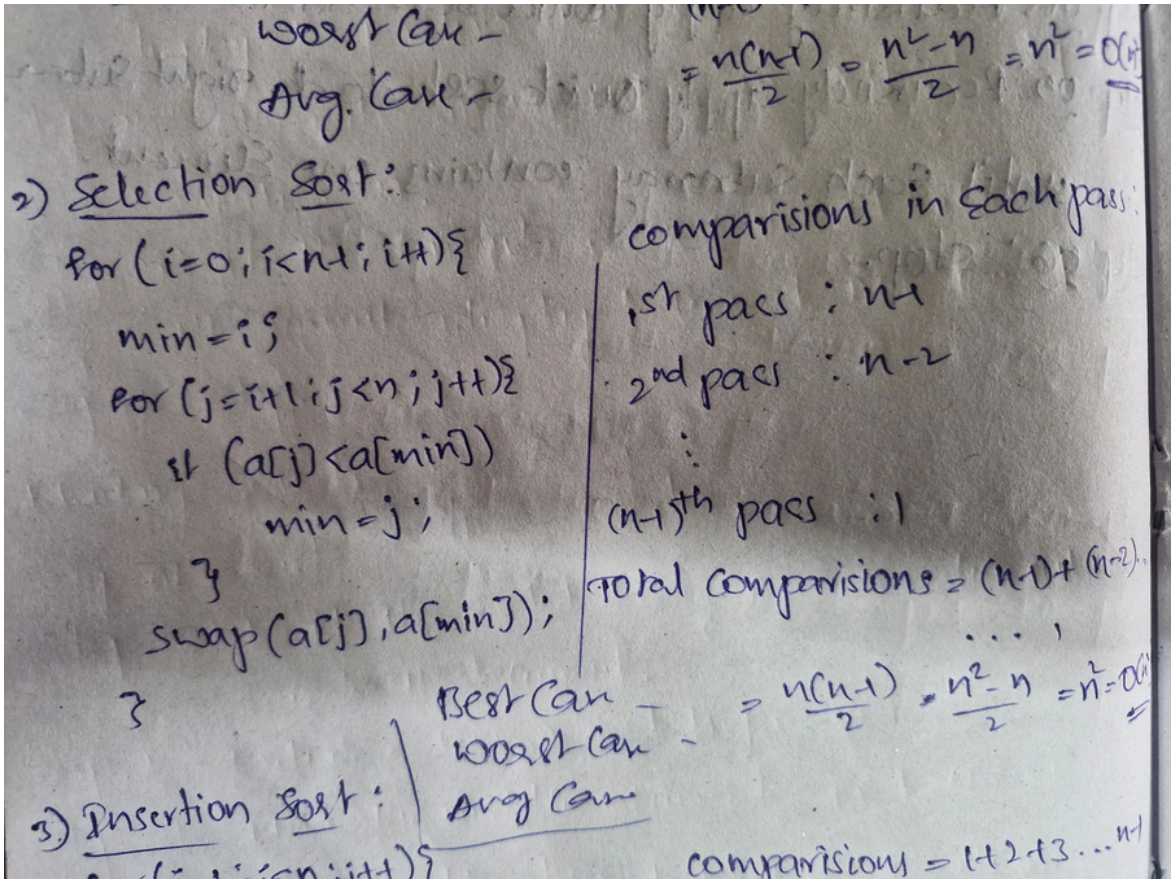
Program:

```
#include <iostream>
#include <vector>

voidselectionSort(std::vector<int>&arr) {
    int n = arr.size();
    for(inti=0;i<n-1;++i){
        int minIndex = i;
        for(intj=i+1;j<n;++j){
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i],arr[minIndex]);
        }
    }
}

int main() {
    int n;
    std::cout<<"Enterthenumberofelements: ";
    std::cin >> n;

    if(n<= 0) {
        return 0;
    }

    std::vector<int> arr(n);
    std::cout<<"Enter"<<n<<"integers: ";
    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

    selectionSort(arr);

    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

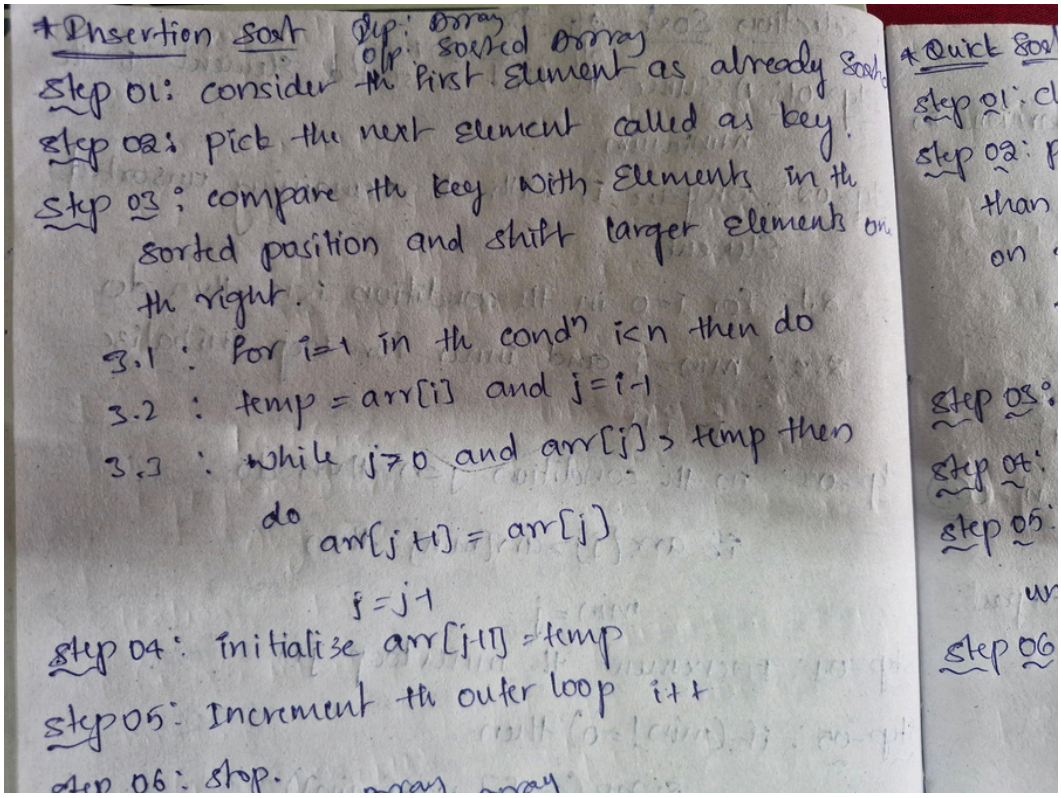
```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 0 1 5 2
Sorted array using  selection sort: 0 1 2 5
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



Insertion sort:

Algorithm:



Program:

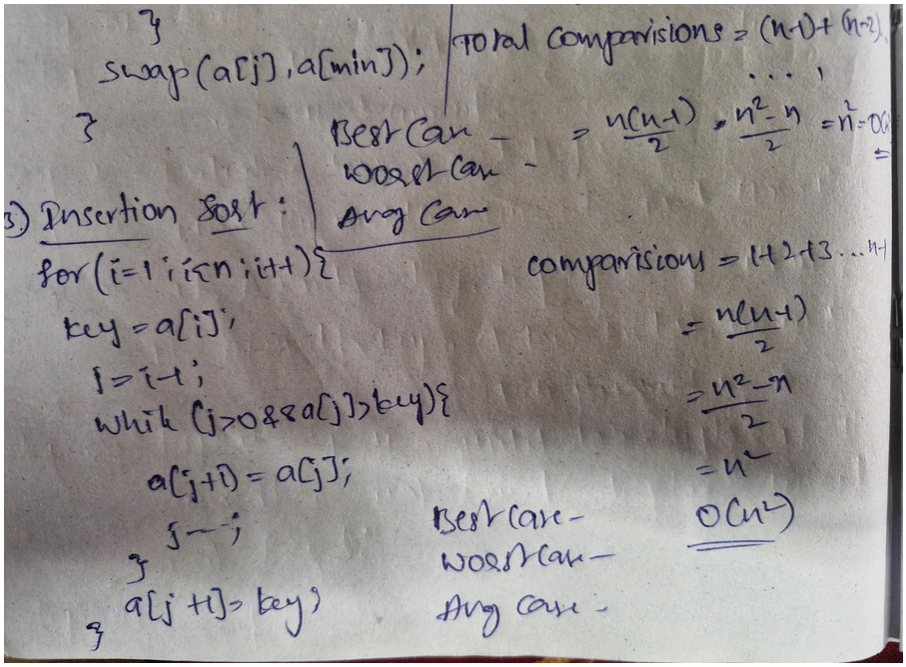
```
#include <iostream>
#include <vector>
voidinsertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for(inti=1;i<n;++i) {
        int key = arr[i];
        intj=i-1;
        while(j>=0&&arr[j] > key) {
            arr[j+1]=arr[j];
            j=j-1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n;
    std::cout<<"Enterthe number of elements: ";
    std::cin >> n;
```

```
    if(n<= 0) {
        return 0;
    }

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";
    for(int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

    insertionSort(arr);

    std::cout << "Sorted array: ";
    for(int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

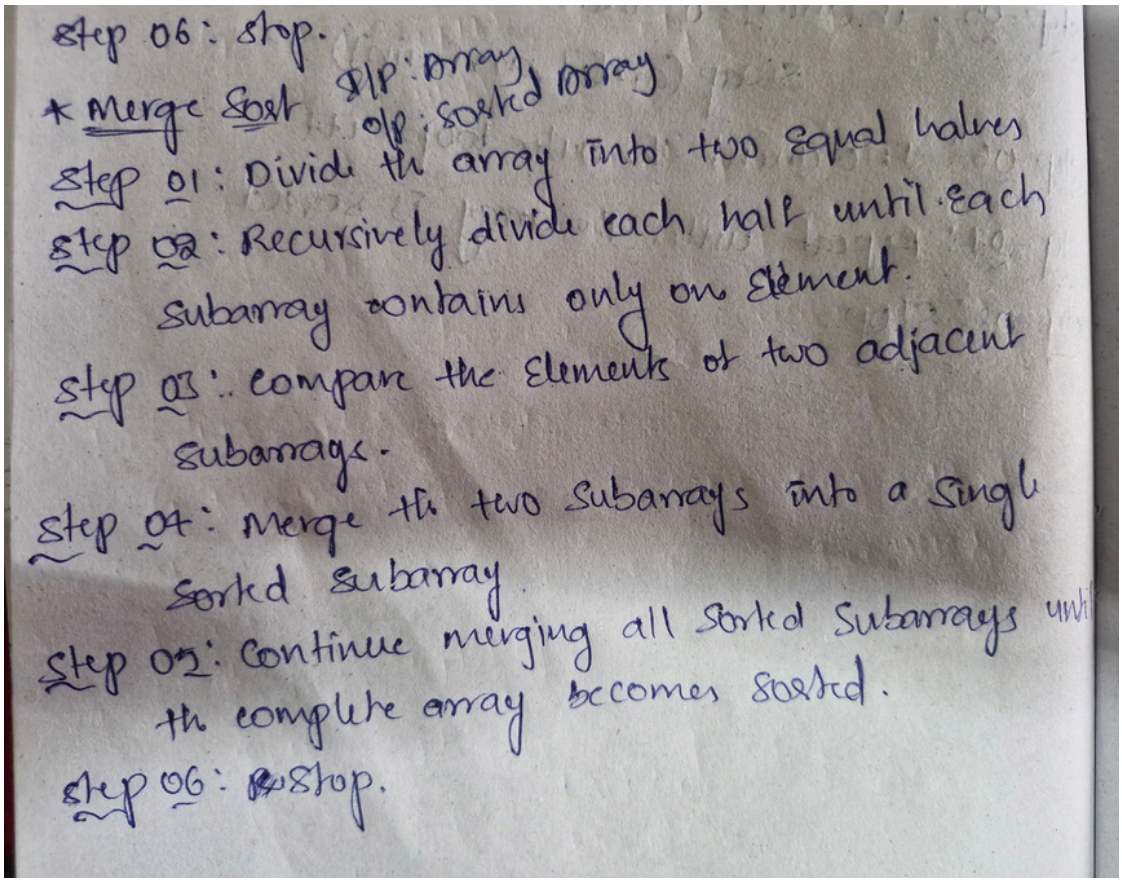
```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 10 2 5 6
Sorted array: 2 5 6 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



***4*. Merge sort:**

Algorithm:



Program:

```
#include <iostream>
#include <vector>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - left;

    std::vector<int>        L(n1);
    std::vector<int> R(n2);

    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }

    int i = 0;
```

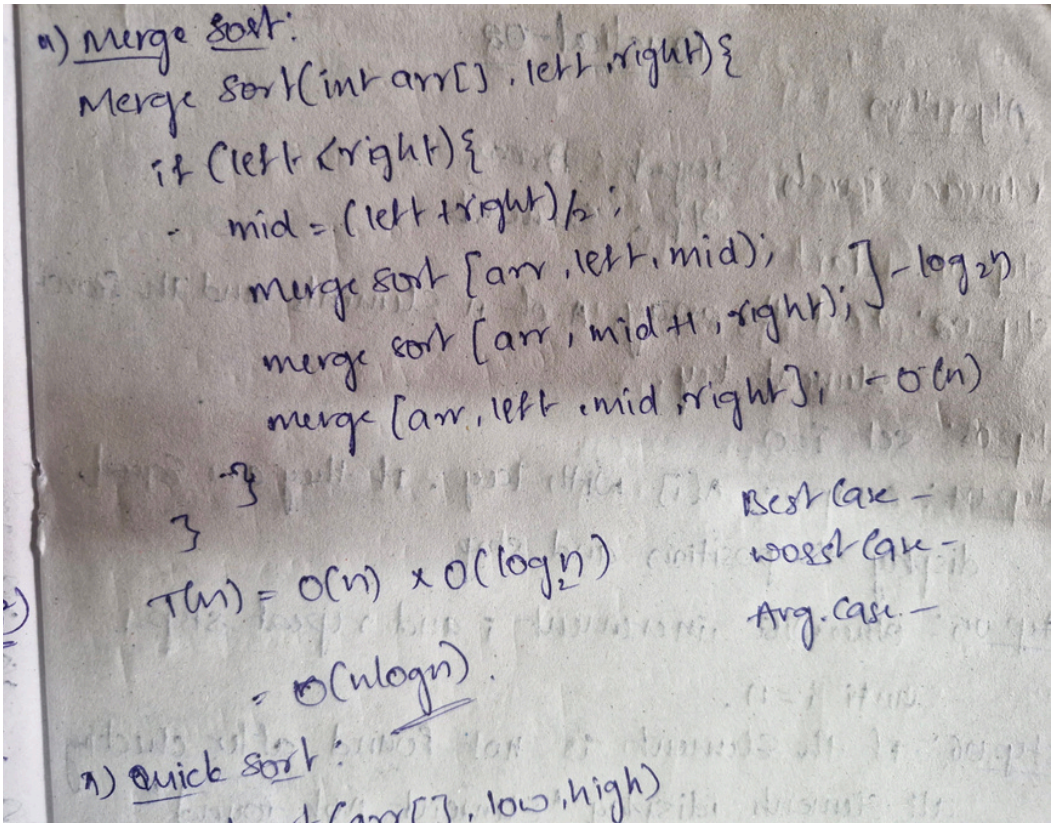
```
    int j = 0;
    int k = left;

    while(i<n1&&j<n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            ++i;
        } else {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }

    while (i < n1) {
        arr[k] = L[i];
        ++i;
        ++k;
    }

    while (j < n2) {
        arr[k] = R[j];
        ++j;
        ++k;
    }
}

voidmergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        intmid=left+(right - left) / 2;

        mergeSort(arr,left, mid);
        mergeSort(arr,mid + 1, right);
        merge(arr,left,mid, right);

    }
}

int main() {
    int n;
    std::cout<<"Enterthe number of elements: ";
    std::cin >> n;

    if (n <= 0) {
    return 0;
    }
```

```
    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";
    for(int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

    mergeSort(arr, 0, n - 1); std::cout

    << "Sorted array: ";
    for(int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

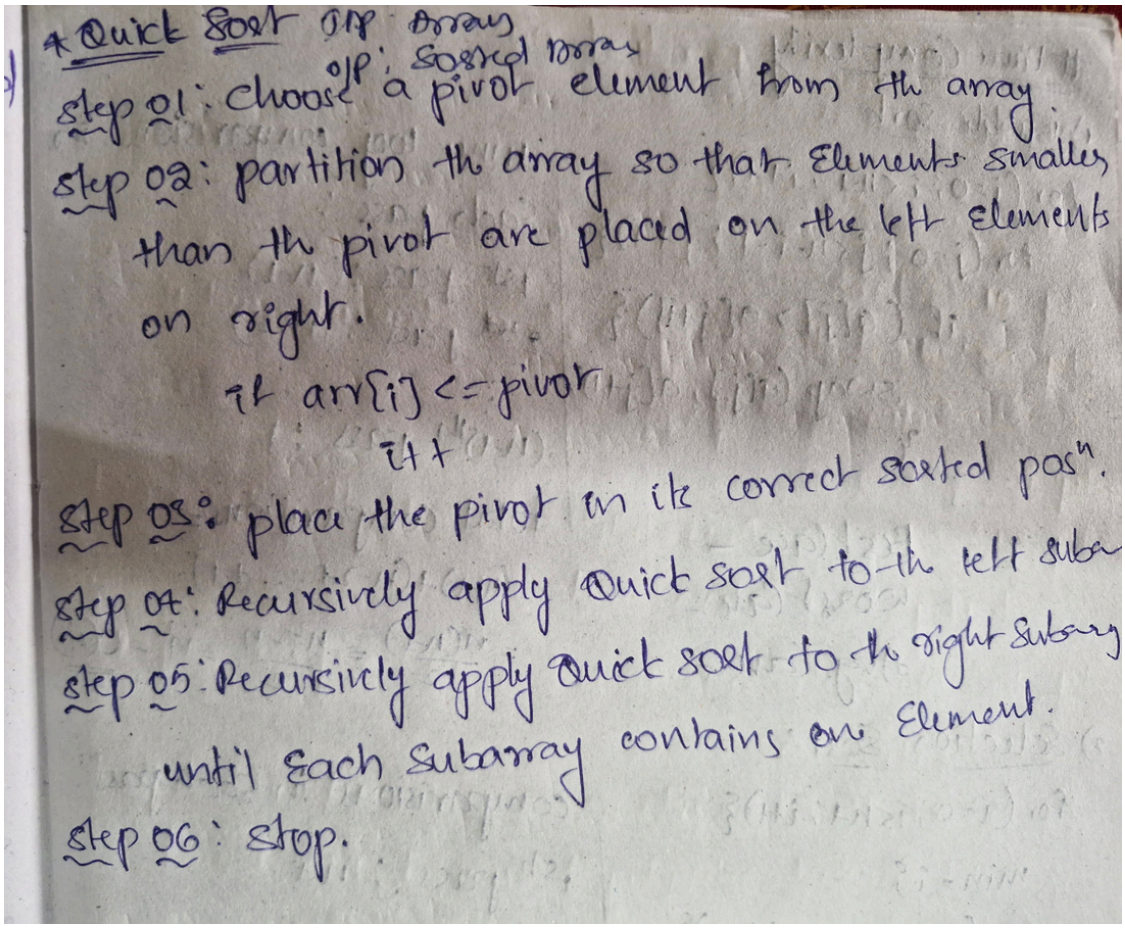
```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 10 2 5 1
Sorted array: 1 2 5 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



5.Quick sort:

Algorithm:



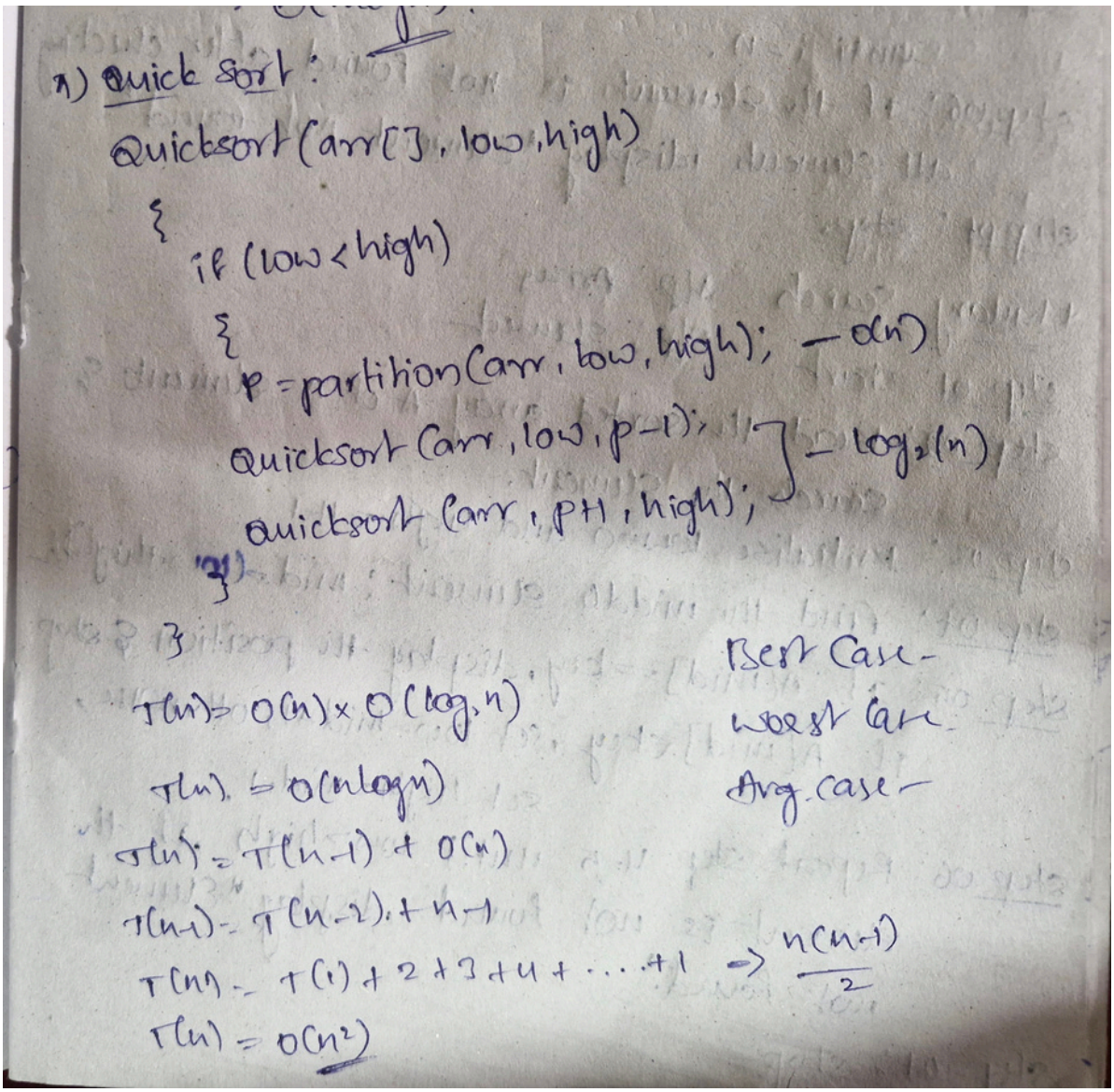
Program:

```
#include <iostream>
#include <vector>
voidswap(int& a, int& b) {
    inttemp = a;
    a=b;
    b=temp;
}

intpartition(std::vector<int>& arr, int low, int high) {
    intpivot = arr[high];
    inti= low - 1;
```

```
    for(intj=low;j< high; j++) {
        if(arr[j]<pivot) {
            i++;
            swap(arr[i],arr[j]);
        }
    }
    swap(arr[i+1],arr[high]);
    return i + 1;
}
voidquickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        intpi=partition(arr, low, high);
        quickSort(arr,low, pi - 1);
        quickSort(arr,pi + 1, high);
    }
}
int main() {
    int n;
    std::cout<<"Enter number of elements: ";
    std::cin >> n;

    std::vector<int>arr(n);
    std::cout<<"Enter " << n << " elements: ";
    for(inti=0;i<n; i++) {
        std::cin>>arr[i];
    }
    quickSort(arr,0,n - 1);
    std::cout<<"Sorted array: ";
    for(inti=0;i<n; i++) {
        std::cout<<arr[i] << " ";
    }
    std::cout<<std::endl;
    return 0;
}
```

Output:

Time complexity:



conclusion :

Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selectionsort, Insertion sort, Merge sort and Quicksort.
The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.